Take-Home Assignment — Supervised Voice AI Agent on Self-Hosted LiveKit

Candidate: Vijay Verma
Time window: 30 hours from when you receive this email
AI copilots: Allowed (Cursor, Claude, Copilot, anything). You must be able to explain every line you submit.
Quality bar: Proof of concept, not production. Working end to end matters more than polish.

1. Context

We build voice AI for contact centres. An AI agent answers a customer call in real time, and a **human supervisor** can listen in, privately guide the agent, or take over. This assignment is a small version of that.
You will build a voice agent on **LiveKit**, running on **your own self-hosted LiveKit server** (not LiveKit Cloud).

2. What to Build

Part A — Self-host LiveKit (required)

- Run your own **LiveKit server** locally with Docker (or on any VM you like). **Do not use LiveKit Cloud.**
- Use your own API key and secret from your server's config.
- In your README, write down which ports you opened and **why each one is needed**. Cover signalling, the TCP fallback and the UDP media range.

Part B — Voice agent (required)

Build a voice agent using the **LiveKit Agents framework in Python**.

- Pipeline: **speech-to-text** → **LLM** → **text-to-speech**, with voice activity detection (VAD) and turn detection.
- Providers are your choice. Free tiers are fine: for example Deepgram, AssemblyAI, Groq or Whisper for speech-to-text; OpenAI, Gemini or Groq for the LLM; ElevenLabs, Cartesia, Deepgram or OpenAI for text-to-speech. Local models are fine too.
- The agent plays a **customer-support persona** for a fictional company. Keep the system prompt in a database, not hard-coded (see Part C).
- **Interruptions (barge-in):** when the user speaks over the agent, the agent must **stop talking immediately**. The agent's conversation history must contain **only what was actually spoken to the user** before the interruption, not the full sentence the LLM generated.

Part C — Backend API (required)

A **FastAPI** backend with:

- POST /agents and GET /agents/{id}: create and fetch an agent config (name, system prompt, voice, model). Store it in SQLite or Postgres.
- POST /calls: start a call for a given agent. Create or choose a room, issue a participant token, and make sure an agent worker joins **that** room with **that** agent's config.
- GET /calls/{id}: return call status, and the transcript once the call ends.

Part D — Web client (required)

A simple browser page (React or plain JS):

- Pick an agent, click **Call**, and talk to it through your mic.
- Show a **live transcript** with a speaker label on every line (agent or user).

Part E — Supervisor (required)

A second browser page for a **supervisor** who joins the **same room**:

- **Listen:** hear both the customer and the agent. The customer must not be able to tell a supervisor is there.
- **Whisper:** type a private instruction, e.g. "offer a 10% discount". Send it over LiveKit's **data channel**. The agent must use it in its next reply **without reading it out and without ever mentioning a supervisor**.

Part F — Post-call compliance check (required)

When a call ends, save the transcript with **timestamps and speaker labels**, then check two rules:

1. *"The agent must say the call is being recorded within the first 30 seconds."* Check this **deterministically from timestamps**. Do not use an LLM for it.
2. *"The agent must not promise a refund."* Check this with an **LLM**, and return structured output (a pass/fail plus the offending line).

Expose the results in GET /calls/{id}.

Part G — Latency measurement (required)

For every agent turn, log how long each stage took, in milliseconds:
user stops speaking → speech-to-text final → LLM first token → text-to-speech first audio → audio published
Include a table of real measured numbers from your own calls in the README.

3. Stretch Goals (optional — pick any two if you have time)

- **Take over:** the supervisor clicks a button, the agent goes silent, and the supervisor speaks directly to the customer. Then hand control back to the agent.
- **Recording:** record the full call (both sides) with **LiveKit Egress** and store it (local disk or S3).
- **Concurrency:** run **2+ agent worker processes** and show 3 simultaneous calls being

shared between them.
- **Everything in docker compose up:** LiveKit server, backend, agent worker(s) and database.

4. What to Submit

A Git repository (GitHub/GitLab, public or shared with us) containing:
- **Source code** for everything above.
- **A README** covering: setup steps a reviewer can follow from scratch; an architecture diagram (Excalidraw or similar is fine); the ports and config for your self-hosted LiveKit, with the reason for each; your measured latency table (Part G); and known limitations and what you would do next.
- **A 3–5 minute screen recording** showing a call, an interruption, a whisper taking effect, and the compliance result.

5. Prepare for the Review Call (Important)

The next round is a **45–60 minute review of your submission**. We will walk through your code, then discuss how to take it to production at scale.
Read the LiveKit and WebRTC material thoroughly before the call. We will go deep on *how things work*, not just on what you used. Be ready to explain:

LiveKit

- What an **SFU** is, and why LiveKit forwards media instead of mixing it
- Rooms, participants, tracks, publishing and subscribing
- How an **agent worker** registers with the server, gets a job, and joins a room (the dispatch model)
- How the agent session handles VAD, turn detection, interruptions and streaming between speech-to-text, the LLM and text-to-speech
- Data channels (reliable vs lossy), and when you'd use each
- Egress (recording) and ingress; SIP for phone calls at a high level
- What changes when you self-host vs use LiveKit Cloud

WebRTC

- The connection lifecycle: signalling, SDP offer/answer, **ICE**, candidate types (host, server-reflexive, relay)
- **STUN vs TURN**: when TURN is needed, and what it costs
- Why media travels over **UDP**, and what jitter, packet loss and jitter buffers do
- The Opus codec, and why phone audio (G.711) differs

Production thinking

- How you'd scale to **10,000 concurrent calls**: agent workers, media servers, what runs out first
- Cost per call-minute, and where most of it goes

- What happens to a live call if an agent worker crashes, or the media server restarts
- Where latency comes from, and which stage you'd attack first

Good starting points: docs.livekit.io (Home → Self-hosting; Agents framework; Realtime / data messages; Egress), the LiveKit Agents GitHub repo and its examples, and any solid WebRTC primer (e.g. webrtcforthecurious.com).

6. How We Evaluate

Area	**What we look for**
Works end to end	Self-hosted LiveKit, agent, API, client, supervisor, compliance check
Understanding	You can explain every part of your code and how LiveKit/WebRTC actually work underneath
Correctness	Barge-in keeps history accurate; the 30-second rule uses timestamps; LLM output is validated
Engineering judgement	Sensible data model, error handling, retries, clear separation of API, agent and storage
Measurement	Real latency numbers, and a reasoned view of where to improve
Production thinking	Scaling, failure handling and cost in the review discussion

Good luck. If anything is unclear, reply to this email and ask; asking good questions is part of the job.